

Introduction to Web Shells

It is almost guaranteed that we will come across web servers in our time learning and actively engaging in the practice of pentesting. Much of the world's software services are moving to web-based platforms accessible over the world wide web using a web browser and HTTP/S. Just consider the website we are on now. It is entirely in the browser, accessible from anywhere in the world using any Internet-connected device. Modern entertainment mediums like **video games**, **music**, and **video streaming** are accessible through browsers and apps. This means we will find ourselves targeting web applications more and more as time goes on.

Furthermore, during external penetration tests, we often find that clients' perimeter networks are well-hardened. They do not expose vulnerable services such as SMB or other elements that we used to frequently encounter. Those elements we now primarily anticipate during an internal penetration test. During our external penetration tests, we most commonly "get in" (gain a foothold inside the internal network) via web application attacks (file upload attacks, SQL injection, RFI/LFI, command injection, etc.), password spraying (against RDS, VPN portals, Citrix, OWA, and other applications using Active Directory authentication), and social engineering.

Web applications are often the majority of what we see exposed during an external network assessment and often present an enormous attack surface. We may find publicly available file upload forms that let us directly upload a PHP, JSP, or ASP.NET web shell. Some functionality during authenticated testing may be present or, our personal favorite, a self-registration functionality where we can go in and upload a web shell (after bypassing client-side checks) in the user profile picture upload area. We may also come across applications such as Tomcat, Axis2, or WebLogic, which allow you to deploy JSP code via a WAR file as part of their functionality. We may even find a misconfigured FTP service that allows file uploads directly to the server's webroot. There are many other ways that we may find to upload a web shell that is outside the scope of this module. What comes next once we have identified an unrestricted upload vulnerability or misconfiguration?

What is a Web Shell?

A **web shell** is a browser-based shell session we can use to interact with the underlying operating system of a web server. Again, to gain remote code execution via web shell, we must first find a website or web application vulnerability that can give us file upload capabilities. Most web shells are gained by uploading a payload written in a web language on the target server. The payload(s) we upload should give us remote code execution capability within the browser. The proceeding sections and challenges will primarily be focused on executing commands through our web shells in the browser. Still, it is essential to know that relying on the web shell alone to interact with the system can be unstable and unreliable because some web applications are configured to delete file uploads after a certain period of time. To achieve persistence on a system, in many cases, this is the initial way of gaining remote code execution via a web application, which we can then use to later upgrade to a more interactive reverse shell.

In the following few sections, we will learn and experiment with various web shells that allow us to interact with a web server's underlying OS through the web browser.

[< Previous](#)[Next >](#)[✔ Mark Complete & Next](#)[📄 Cheat Sheet](#)

Table of Contents

Introduction

Shells look like this. Payloads deliver the shells.

Shells Jack US III, Payloads Deliver US Shells



CAT5 Security's Engagement Preparation



Shell Basics

 Anatomy of a Shell



 Bind Shells



 Reverse Shells



Payloads

Introduction to Payloads



 Automating Payloads & Delivery with Metasploit

Crafting payloads with MSFvenom



Windows Shells

 Infiltrating Windows



NIX Shells

 Infiltrating Unix/Linux



Spawning Interactive Shells



Web Shells

[Introduction to Web Shells](#)

 Laudanum, One Webshell to Rule Them All

 Antak Webshell

 PHP Web Shells

Skills Assessment

 The Live Engagement

Additional Considerations

Detection & Prevention

My Workstation

OFFLINE

 Start Instance

 / 1 spawns left